Hierarchical Patch Merging.

A plain ViT keeps one resolution from input to output. Swin instead builds a *pyramid*, like a CNN: between stages it merges every 2×2 group of neighbouring patches and projects $4C \to 2C$, halving spatial resolution while doubling channels. Token count falls $4\times$ per stage; this multi-scale feature map is exactly what lets Swin serve as a general vision *backbone* for detection and segmentation. Below we derive the resolution and channel schedule, the projection shapes, and count every token. Every number is computed in full.

THE EQUATION

$$(h, w, C) \xrightarrow{\text{concat } 2\times2} \left(\tfrac{h}{2}, \tfrac{w}{2}, 4C\right) \xrightarrow{W \in \mathbb{R}^{4C\times 2C}} \left(\tfrac{h}{2}, \tfrac{w}{2}, 2C\right)$$

Worked example. Swin-T schedule from $56 \times 56 \times 96$: stages $56 \to 28 \to 14 \to 7$, channels $96 \to 768$, tokens $3136 \to 49$; projection matrices $384 \to 192$, $768 \to 384$, $1536 \to 768$.

Read top to bottom: each step feeds the next.

Contents

1 The claim: a CNN-like pyramid built from attention

A plain ViT tokenises once and keeps the same number of tokens at the same resolution through every layer — a *single-scale* representation. Swin instead reduces resolution in stages, the way a CNN's pooling builds a feature pyramid. The operation that does this is **patch merging**: take every non-overlapping 2×2 group of neighbouring patches, concatenate their channel vectors, and linearly project back down.

$$\underbrace{[x_{00}; x_{01}; x_{10}; x_{11}]}_{\in \mathbb{R}^{4C}} \xrightarrow{W,\, W \in \mathbb{R}^{4C \times 2C}} \underbrace{y}_{\in \mathbb{R}^{2C}}, \qquad (h, w, C) \to \left(\tfrac{h}{2}, \tfrac{w}{2}, 2C\right) \tag{1}$$

Each merge therefore does two things at once:

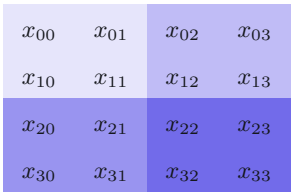
- **Halves** each spatial dimension: $h \to h/2$, $w \to w/2$, so the token count $hw \to hw/4$ drops by $4\times$.
- **Doubles** the channel dimension: $C \to 2C$ (the concat gives $4C$, the projection keeps $2C$).

Symbol	Shape	Meaning
$h \times w$	—	patch grid at the current stage
C	—	channels per token at the current stage
2×2 group	4 patches	a local neighbourhood to be merged
concat	$\mathbb{R}^{4C}$	the four patches' channels stacked
W	$4C \times 2C$	learned merge projection (no bias, after LayerNorm)
y	$\mathbb{R}^{2C}$	merged token, one per 2×2 group

Think of zooming out on a map. Four neighbouring patches (a fine 2×2 tile) become one coarser "super-patch". You see less spatial detail but pack more meaning into each token's richer channel vector. Stacking these zoom-outs gives a stack of maps at different scales — fine maps localise edges, coarse maps recognise whole objects. That is precisely the pyramid a detector or segmenter wants.

2 The 2×2 merge, made concrete

On a small grid, merging groups the patches as below: each colour is one 2×2 group that collapses to a single output token. A 4×4 grid of 16 tokens becomes a 2×2 grid of 4 tokens.



Each shaded 2×2 block $\rightarrow$ one token. Within a block the four C-vectors are *concatenated* (order top-left, top-right, bottom-left, bottom-right) into a $4C$-vector, then $W \in \mathbb{R}^{4C \times 2C}$ projects it to $2C$. Four input tokens, one output token: the $4\times$ reduction.

3 Worked example: the Swin-T schedule

Swin-Tiny starts after patch embedding with a 56×56 grid of $C = 96$-dim tokens. Three merges produce four stages. Token counts: $56^2 = 3136$, $28^2 = 784$, $14^2 = 196$, $7^2 = 49$.

Stage	Resolution	Channels C	Tokens hw	Merge into next ($4C \rightarrow 2C$)
1	56×56	96	3136	$384 \rightarrow 192$
2	28×28	192	784	$768 \rightarrow 384$
3	14×14	384	196	$1536 \rightarrow 768$
4	7×7	768	49	— (final)

Read across: resolution halves ($56 \rightarrow 28 \rightarrow 14 \rightarrow 7$), channels double ($96 \rightarrow 192 \rightarrow 384 \rightarrow 768$), tokens fall $4\times$ each step ($3136 \rightarrow 784 \rightarrow 196 \rightarrow 49$). The factors are exact: $3136/4 = 784$, $784/4 = 196$, $196/4 = 49$.

3.1 Projection shapes and parameter counts

The merge at the end of stage with channels C takes concat-dim $4C$ to output $2C$, so its weight is $W \in \mathbb{R}^{4C \times 2C}$ with $4C \cdot 2C = 8C^2$ parameters (bias omitted; a LayerNorm on the $4C$ input adds $2 \cdot 4C$ tiny params):

After stage	W shape ($4C \times 2C$)	$8C^2$ weights	LN params ($2{\cdot}4C$)
1 (C=96)	384×192	73,728	768
2 (C=192)	768×384	294,912	1,536
3 (C=384)	1536×768	1,179,648	3,072

Check: $8 \cdot 96^2 = 73{,}728$, $8 \cdot 192^2 = 294{,}912$, $8 \cdot 384^2 = 1{,}179{,}648$. Each merge's weight cost quadruples because C doubles ($8(2C)^2 = 4 \cdot 8C^2$).

One merge does "4× fewer tokens, 2× wider channels" in a single learned step. The $4C \to 2C$ projection is the linchpin: concatenation alone would grow channels to $4C$ and blow up downstream cost, so the projection immediately tames it back to $2C$. Net effect per stage: tokens ÷4, channels ×2 — the canonical CNN-style trade of space for depth.

4 Why a backbone needs this: vs. a flat ViT

A flat ViT holds its resolution fixed, so it emits a *single* feature map. Detectors and segmenters need features at *several* scales (small objects from fine maps, large objects from coarse maps) — Swin's four stages hand them exactly the $\{1/4, 1/8, 1/16, 1/32\}$-resolution maps a Feature Pyramid Network expects. Swin also processes *fewer tokens overall* because later stages are tiny:

Model	Tokens per stage / layer	Total tokens across the 4 scales
Swin (pyramid)	3136, 784, 196, 49	**4165**
Flat ViT (single scale)	3136 at every stage	$4 \times 3136 = 12{,}544$

Summed over the four scales, Swin handles 4165 tokens against a flat ViT's 12,544 for the same number of stages — about 3× fewer — *and* delivers multi-scale outputs the ViT cannot. Combined with windowed attention (Topic 1), this is why Swin is a drop-in vision backbone where ViT is not.

Patch merging is **not** pooling. Average- or max-pooling would discard or pick among the four patches with *no parameters*; merging *concatenates* all four full C-vectors and applies a *learned* $4C \to 2C$ matrix. So it mixes information across the four patches and across all $4C$ input channels — it can learn, for example, to keep an edge from the top-left patch and texture from the bottom-right. Treating it as parameter-free downsampling both miscounts the model size (you would miss the $8C^2$ weights per merge) and misunderstands what it learns.

5 Where this sits in the track

Component	Role	Where
Windowed / shifted attention	linear-cost attention with cross-window flow	Topic 1
Patch merging	builds the $\{56, 28, 14, 7\}$ multi-scale pyramid	this topic
Swin block $\times$ stages	W-MSA + SW-MSA between merges	this module
ViT (single scale)	the contrast: one resolution, no pyramid	Phase 6, ViT

Together with shifted-window attention, hierarchical patch merging completes the Swin recipe: a transformer that scales linearly with image area *and* produces the CNN-style feature pyramid downstream vision tasks rely on.

A Reproducible (NumPy)

Inputs: start $56 \times 56 \times 96$ (Swin-T), three 2×2 merges, projection $4C \to 2C$.

```
import numpy as np
h = w = 56; C = 96
for stage in range(1, 5):
    print(stage, f"{h}x{w}", C, h*w)          # resolution, channels, tokens
    if stage < 4:
        print("   merge W:", 4*C, "->", 2*C, " params 8C^2 =", 8*C*C)
        h //= 2; w //= 2; C *= 2
# 1 56x56 96 3136 | merge 384->192 params 73728
# 2 28x28 192 784 | merge 768->384 params 294912
# 3 14x14 384 196 | merge 1536->768 params 1179648
# 4 7x7 768 49

tokens = [3136, 784, 196, 49]
print(sum(tokens), 4*3136)                     # 4165 (Swin) vs 12544 (flat ViT, 4 stages)
```